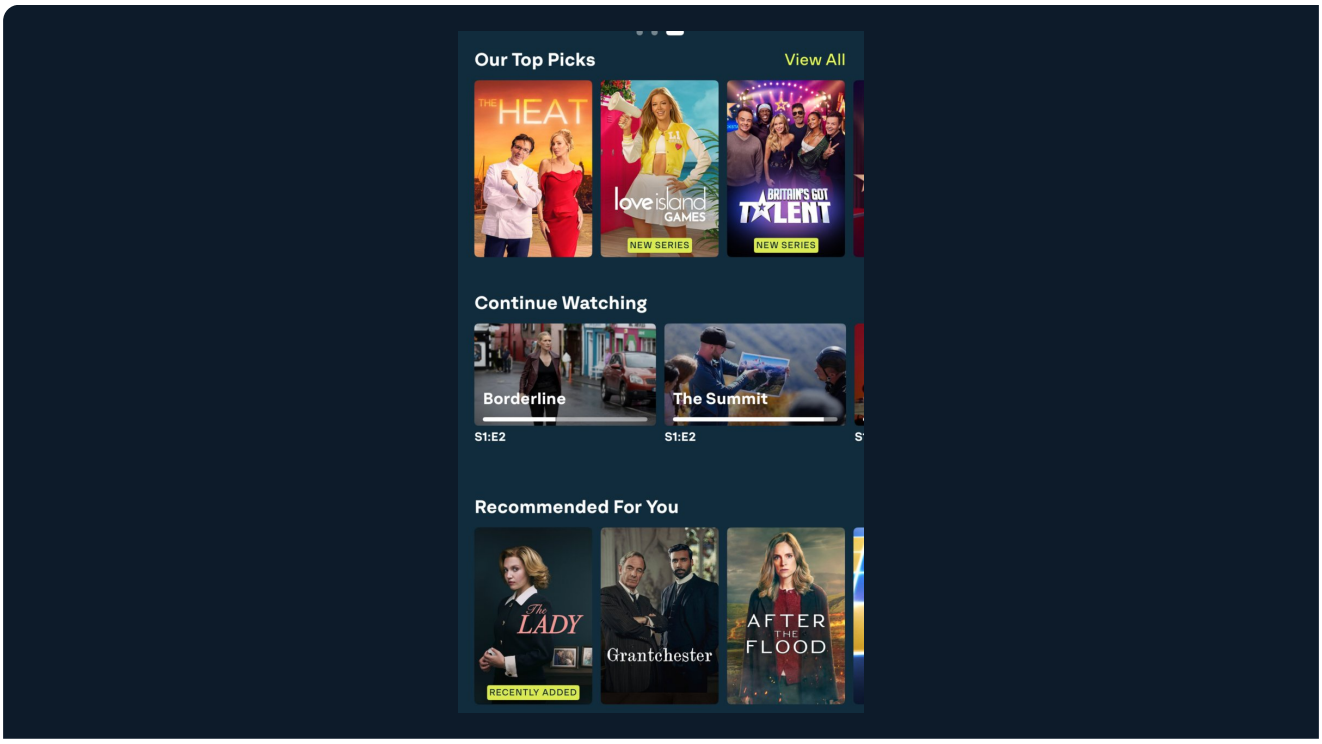
# iOS Developer

Technical Test

## Overview

Build a small iOS application that consumes a REST API and renders a homepage layout similar to a modern streaming application. The task is designed to be completed within a **4–6 hour timebox** and should demonstrate your approach to architecture, code quality, and real-world iOS development practices.




Example homepage layout – streaming application UI

## API Endpoint

The application must consume the following REST endpoint. The response contains all data required to populate the homepage sections.

| | |
|---|---|
| **Method** | `GET` |
| **URL** | `https://api.jsonbin.io/v3/b/69df608e36566621a8b675e1` |
| **Auth** | `None required` |
| **Response** | `JSON – array of content rail objects` |

## Core Requirements

| | |
|---|---|
| **Framework** | UIKit or SwiftUI — both are acceptable. |
| **UI Construction** | All UI must be built programmatically. Storyboards, XIB and NIB files are not permitted. |
| **Data Rendering** | Render the homepage using the data returned from the API response. |
| **Layout** | Each rail section should scroll vertically; items within each rail scroll horizontally. |
| **Aspect Ratios** | Handle both 2×3 portrait and 16×9 landscape image aspect ratios correctly. |
| **Content Types** | Support the following types: brand, episode, series, film, page. |
| **Image Loading** | Load images asynchronously with appropriate caching — do not block the main thread. |
| **Resilience** | Handle optional and missing fields gracefully without crashing. |

## Architecture Expectations

| | |
|---|---|
| **Pattern** | Use a clear architectural pattern — MVVM is preferred. |
| **Separation** | Separate networking, decoding, and UI concerns into distinct layers. |
| **Parsing** | Use Swift Codable for all JSON decoding. |
| **Dependencies** | Demonstrate clean, considered dependency management. |

## Bonus — Optional Enhancements

The following items are entirely optional but will be viewed favourably during review:

- Loading and error state handling (e.g. skeleton screens or retry UI).
- Pull-to-refresh functionality.
- Basic unit tests covering networking and/or view model logic.

## Submission Guidelines

- Share a **GitHub repository** containing your completed project.
- Include a clear **README** with build and run instructions.
- Document any **assumptions** made during development.
- Please **timebox the task to 4–6 hours**; quality over quantity is preferred.

## What We Will Be Looking For

| | |
|---|---|
| **Code Quality** | Readability, consistency, and adherence to Swift best practices. |

| | |
|---|---|
| **Architecture** | Clarity of design decisions and separation of concerns. |
| **Edge Cases** | Correct handling of optional data, missing fields, and error conditions. |
| **UI Layout** | Visual correctness, adaptability across device sizes, and aspect ratio handling. |
| **Swift Practices** | Use of modern Swift idioms, concurrency patterns, and API conventions. |

**Note:** We are interested in how you approach the problem, not just the final output. Clear reasoning, considered trade-offs, and honest README notes are all valued.